Finding laundering on a public ledger

Team XRPLarp · Decode & Detect hackathon, 24 October 2026 · Method write-up for version 12 (final) · blend of versions 9 (73.4) and 11 (73.3) · metric: 100 × average precision

1. The problem in plain terms

We were given one week of payments on a fictional copy of the XRP Ledger: about 40,000 payments to learn from (each marked suspicious or not) and another 40,101 to score. Every payment shows who sent money to whom, how much, in which currency and exactly when. It does not show who is behind each wallet or why they paid.

Our job was to give each test payment a score from 0 to 1 so that the suspicious ones end up at the top of the list, the way an anti-money-laundering analyst's review queue should look. Only the order matters: the score is the average precision of that ranking.

> **The main difficulty.** Laundering schemes in the data have honest twins. For every scheme shape (money collected from many wallets, split and re-merged, sent round in a loop, passed through rows of middlemen) there are innocent look-alike groups with the same structure, such as payroll, exchanges and shops. In our tests, structure and amounts alone told the two apart no better than a coin toss. What separates them is mostly *timing* and *where the money came from*.

2. How the detector works

1. Tidy	2. Describe	3. Link	4. Score	5. Share
One clean row per payment, valued in US dollars	About 400 numbers per payment about its behaviour and neighbours	A model guesses which payments belong to the same scheme	Two tree-model ensembles rate each payment	Linked payments share suspicion, then the final score

Step 1: Tidy the payments

Each transaction can have up to three money "legs", for example when one currency is converted into another. We turn it into one row: sender, receiver, asset delivered, what was really paid (the first leg, not the stated maximum), and the value in US dollars using the organisers' hourly price for that exact asset. A currency is identified by both its code and its issuer, so "USD from issuer A" and "USD from issuer B" are treated as different assets.

Step 2: Describe every payment with numbers

A model cannot look at a payment the way a person does, so we describe each one with about 400 measurements. The most useful families are:

- **Is this a new relationship?** How often this sender has paid this receiver during the week. Planted schemes almost always use first-time pairs.
- **Bursts.** How busy the sender and receiver are in windows from 2 minutes to 8 hours around the payment.
- **Local shape.** Payments that share a wallet and happen close together are joined into small groups. We measure each group's size, number of wallets, how long it lasts, loops, number of currencies and how evenly spaced its payments are.
- **Money moved on.** Did the receiver pass similar amounts onward within a few hours, or split them up or merge them with others?
- **Timing regularity** (new in version 8). Planted schemes run on a schedule, so the gaps between their payments are more regular than real life. We measure how much those gaps

vary.

Step 3: Work out which payments belong together

A laundering scheme is many payments working together, and all of them share the same label. So we trained a separate **link model** that looks at two payments touching the same wallet within 8 hours and estimates the probability that they belong to the same scheme. It uses the time gap, direction, amounts, currencies and how unusual the pair of wallets is. On held-out training schemes it reaches 0.99 AUC (near-perfect at telling linked from unlinked pairs).

Some "relay" schemes repeat the same payment between the same two wallets in rounds days apart, far beyond 8 hours. So the link model also connects each payment to the next one or two payments between the same pair.

These links then produce further measurements:

- **Rebuilt groups.** Confidently linked payments are grouped, and each payment learns its group's size, duration, number of middlemen and how much money passes straight through.
- **Was the money there first? ("soft funding")** A real laundering middleman only pays out money it has already received. An innocent look-alike often pays out before money arrives. For each payment we add up the probable same-scheme money that arrived *before* it versus only *after* it, weighting every link by its probability.
- **Neighbours' evidence** (new in version 8). Each payment also takes on the probability-weighted average of key measurements from the payments it is linked to, including links to earlier rounds days before. This is how "the money was there first" evidence crosses multi-day gaps.

Step 4: Score with two models

We use gradient-boosted decision trees, a standard and robust method for table data. Each model is an average of LightGBM and XGBoost, each trained with 5 random seeds, so no single run decides the result.

- **Model A** answers the real question: suspicious or not?
- **Model B** answers an easier one: does this payment belong to any scheme-shaped group, laundering or innocent look-alike? With twice as many examples it learns what slow, spread-out schemes look like, which helps Model A on the hard cases.

Step 5: Let linked payments share suspicion

Because all payments in a scheme share a label, each model's score is blended (15% own score, 85% neighbours) with the average score of its linked payments, weighted by link probability, and with the average over its short bursts of first-time payments. The final score is **Model A + 0.25 × Model B**, rescaled to 0–1. Scores are continuous, so there are no ties.

3. Why it works: probabilities beat hard groups

Our biggest lesson: every improvement since version 5 came from replacing a hard yes/no grouping with a probability-weighted one.

Version	Hard version replaced	Soft replacement	Gain
v6	Average score over groups linked at ≥ 90% confidence	Average weighted by each link's probability	+0.8
v6.1	Clusters with a yes/no "was it funded?" test	Probability-weighted funding sums, no clusters	+0.7
v8	Measurements of a fixed group	Probability-weighted measurements of linked payments, across rounds	+0.7

The reason is measurable. A middleman wallet in a relay often serves two or three schemes at once (78% of its consecutive payments switch scheme), so any hard group is only about 80% pure. The key test, "never pays out more than it has received", is broken by a single wrong payment. Probability weights degrade gracefully where hard groups break.

4. Final step: blending two versions (version 12)

From version 8 we built two improved versions on the same pipeline. **Version 9** (leaderboard 73.4) adds soft running balances (how much probable same-scheme money a wallet holds over time), each wallet's balance trajectory, and neighbour evidence split by link type. **Version 11** (73.3) adds second-hop neighbour evidence: what the neighbours of a payment's neighbours look like.

Because they add different evidence, they make partly different mistakes, so combining them should help. Our final score converts each version's scores into ranks (1st, 2nd, 3rd ... divided by the number of payments) and averages the two with equal weight. We use ranks because only the order matters for the metric, and the two versions use different scales for low scores, so a raw average would let one count more than the other. Equal weights reflect their near-identical leaderboard scores. The blend was not tuned or validated locally.

5. How we checked it

Before uploading we measured the score ourselves with **grouped 5-fold cross-validation** on the training data: we held out a fifth of the schemes, trained on the rest, and repeated this five times. Every payment of a scheme stays in the same fold, so the model is never tested on a scheme it has partly seen. Our local scores are about 10 points below the leaderboard but ranked the versions in the same order.

Version	Main addition	Local AP	Leaderboard
v1	Model A, LightGBM only	–	65.5
v2	+ Model B (scheme shape)	–	67.1
v3	+ XGBoost in the average	–	67.4
v4	+ link model and rebuilt groups	61.7	70.3
v5	+ cross-round links, per-wallet groups	61.5	71.2
v6 / v7	+ soft funding, probability-weighted sharing	62.3	72.0 / 72.4
v8	+ neighbours' evidence, timing regularity	62.8 ± 0.6	73.1
v9 / v11	+ soft balances / second-hop neighbours	–	73.4 / 73.3
v12	**rank blend of v9 and v11 (submitted)**	–	**pending**

By difficulty (local AP for v8): easy schemes 50.5, medium 9.7, hard 2.6. About 81% of our top 1% of payments are truly suspicious. The model catches most easy cases and some medium ones. The hard multi-round relay cases are still essentially missed.

6. What did not work

- **Structure and amount features** aimed at telling schemes from their look-alikes (amount conservation, loops, layer order, density, round numbers, currency changes, fees): all about 0.50–0.52 AUC, no better than chance.
- **A dedicated model for relay schemes:** only 12 such schemes in training, too few to learn from (0.48 AUC).
- **Stricter clustering methods:** either impure groups or groups that missed most of the scheme.

- **Blend tweaks** (other weights, a "rank-aware" blend): neutral or slightly worse.

7. Limitations

- The data is synthetic. A high score here does not show the method works on real financial crime, and this is not a certified compliance tool.
- About one in five top-ranked payments is a false alarm, and hard relay schemes are mostly missed, so results need human review.
- Several signals (first-time pairs, regular timing, night-time activity) reflect how this data was generated. On real data they could unfairly flag new users or legitimate automated services.
- The weights 0.25 and 0.85 were set by hand. Learning them, adding more seeds or a third tree library, and softening the remaining hard groups are the next steps.

8. Data, labels, AI use and reproducibility

- **Data:** only the organisers' files. No outside data, no pre-trained models, no online services at prediction time.
- **Labels:** "suspicious" is Model A's target. The scheme ID trains the link model and keeps schemes together in validation. The scheme family defines Model B's target only. No label is ever a model input, and none is used to score the test data.
- **AI use:** the code and analysis were written with Claude Fable and Claude Opus 5.5 (Anthropic) and GPT Astra (OpenAI), default settings, at our team's direction during the event. The team chose the approach, reviewed the results and is responsible for the submission. Model, settings and prompts are listed in the README and `PROMPTS.md`.
- **Reproduce:** `pip install -r requirements.txt` then `python predict.py --train train_pkg/train --test test_pkg --out predictions.csv`. Runs both versions and blends them: about 40 minutes on a laptop CPU, Python 3.12, fixed seeds (0–4), MIT licence. Blend only: `python blend.py --a predictions_v9.csv --b predictions_v11.csv --out predictions.csv`.

Ideas drawn from published work: FlowScope (Li et al., AAAI 2020), Savage et al. (2016), Weber et al. (2019), Elliptic2 (Bellei et al., 2024), Jullum et al. (2020), IBM AMLworld (Altman et al., NeurIPS 2023). No code from these works was used.